

Neural Networks Overview

Connection to Logistic Regression

- **Logistic Regression Recap:** The model follows a specific computation graph:
 - Input features (x) and parameters (w, b) are used to compute z .
 - z is used to compute the activation a (interchangeable with prediction $\hat{y}$).
 - Finally, the Loss function (L) is computed.
- **Neural Network Definition:** A neural network is essentially constructed by stacking multiple **sigmoid units** together.
 - It repeats the logistic regression process: performing a z calculation followed by an a calculation multiple times in sequence.

Notation Standards

It is crucial to distinguish between layer indices and training example indices:

- **Superscript Square Brackets $[l]$:** Refer to quantities associated with a specific **layer** of the neural network.
 - *Example:* $z^{[1]}$ denotes the z -value calculated in the first layer.
- **Superscript Round Brackets (i) :** Refer to the i^{th} individual **training example**.
 - *Example:* $x^{(i)}$ denotes the input features for the i^{th} training example.

Neural Network Architecture (2-Layer Example)

The computation flow is divided into layers:

1. Layer 1:

- Takes input features x and parameters (w, b) to compute $z^{[1]}$.
- Computes activation $a^{[1]}$ (e.g., using sigmoid function).

2. Layer 2:

- Uses the output of Layer 1 ($a^{[1]}$) to compute a new linear step $z^{[2]}$.
- Computes the final activation $a^{[2]}$, which serves as the final output $\hat{y}$.

Computation Graph

- **Forward Propagation:** The calculation flows from input to output ($z^{[1]} \rightarrow a^{[1]} \rightarrow z^{[2]} \rightarrow a^{[2]}$) to eventually compute the loss.
 - **Backward Propagation:** Similar to logistic regression, derivatives are computed in a **right-to-left** direction (backward calculation).
 - This involves computing gradients (e.g., $da^{[2]}, dz^{[2]}$) to eventually find parameter updates (dw, db).
-

Neural Network Representation

Network Architecture Components

A neural network is typically visualized as a stack of layers comprising individual nodes (units).

- **Input Layer:** The first layer containing the input features ($x_1, x_2, x_3 \dots$).
 - Vertical stack of input values.
 - Passes values x to the next layer.
- **Hidden Layer:** The intermediate layer(s) between input and output.
 - **Why "Hidden"?** In supervised learning, the training set contains inputs (x) and target outputs (y). The values computed by nodes in the middle layers are *not* observed in the training set; they are internal to the model.
- **Output Layer:** The final layer responsible for generating the predicted value $\hat{y}$.

Notation and Activations

We use specific notation to track values across different layers. The term **Activation** (a) refers to the values a layer passes to the next layer.

- **Layer Indexing:**
 - **Input Layer ($a^{[0]}$):** The input features x are denoted as the activations of layer 0.
 - $a^{[0]} = x$
 - **Hidden Layer ($a^{[1]}$):** The activations generated by the first hidden layer.
 - If this layer has 4 nodes, $a^{[1]}$ is a 4×1 column vector.
 - Individual nodes are denoted as $a_1^{[1]}, a_2^{[1]}$, etc.

- **Output Layer ($a^{[2]}$):** The final value generated by the network.
 - In a standard binary classification, this is a real number.
 - $a^{[2]} = \hat{y}$

Layer Counting Convention

- **2-Layer Neural Network:** The example described (Input $\rightarrow$ Hidden $\rightarrow$ Output) is officially called a "2-Layer Neural Network".
- **Rule:** When counting layers, **do not count the input layer** (Layer 0). You only count layers that have associated parameters (Hidden layers + Output layer).

Parameters (w and b)

Each layer (excluding input) has associated parameters used to compute its values.

- **Superscripts:** $w^{[1]}$, $b^{[1]}$ refer to parameters for Layer 1; $w^{[2]}$, $b^{[2]}$ for Layer 2.
 - **Dimensions (Example):**
 - Given: 3 Input features, 4 Hidden units, 1 Output unit.
 - **Layer 1 Parameters:**
 - $w^{[1]}$: (4×3) matrix. (Rows = current layer units; Cols = input features).
 - $b^{[1]}$: (4×1) vector.
 - **Layer 2 Parameters:**
 - $w^{[2]}$: (1×4) matrix. (Rows = current layer units; Cols = previous layer units).
 - $b^{[2]}$: (1×1) vector.
-

Computing a Neural Network's Output

Node-Level Computation

A neural network can be viewed as logistic regression repeated multiple times. Each individual node in a hidden layer performs two distinct steps of computation, exactly like a logistic regression unit:

1. **Linear Calculation:** Compute $z = w^T x + b$.

2. **Activation:** Compute $a = \sigma(z)$.

Notation Breakdown

- $z_i^{[l]}$: The z -value for the i -th node in the l -th layer.
 - $a_i^{[l]}$: The activation value for the i -th node in the l -th layer.
 - Example: $a_1^{[1]}$ refers to the activation of the first node in the first hidden layer.
-

Vectorizing the Computation

Implementing these calculations using a `for` loop for each node is inefficient. Instead, we **vectorize** the equations by stacking the parameters and values into matrices and vectors.

1. Stacking Weights (W)

For a hidden layer with 4 nodes, we have 4 distinct weight vectors (w_1, w_2, w_3, w_4). We stack their transposes to form a matrix $W^{[1]}$:

$$W^{[1]} = \begin{bmatrix} -(w_1^{[1]})^T \\ -(w_2^{[1]})^T \\ -(w_3^{[1]})^T \\ -(w_4^{[1]})^T \end{bmatrix}$$

- **Dimensions:** If there are 3 input features and 4 hidden nodes, $W^{[1]}$ is a (4×3) matrix.

2. Computing z (Vectorized)

By multiplying Matrix $W^{[1]}$ by the input vector x and adding the bias vector $b^{[1]}$, we compute all z values for the layer simultaneously:

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

- **Result:** $z^{[1]}$ is a (4×1) column vector containing $[z_1^{[1]}, z_2^{[1]}, z_3^{[1]}, z_4^{[1]}]^T$.

3. Computing a (Vectorized)

Apply the sigmoid function element-wise to the vector $z^{[1]}$:

$$a^{[1]} = \sigma(z^{[1]})$$

- **Result:** $a^{[1]}$ is a (4×1) column vector.
-

Summary of Neural Network Equations

To compute the output of a 2-layer neural network (1 hidden layer + 1 output layer), you only need **four lines of code** (or equations):

Layer 1 (Hidden Layer)

1. **Linear Step:** $z^{[1]} = W^{[1]}x + b^{[1]}$
2. **Activation:** $a^{[1]} = \sigma(z^{[1]})$

Layer 2 (Output Layer)

The output of Layer 1 ($a^{[1]}$) becomes the input for Layer 2. 3. **Linear Step:** $z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$ 4. **Activation:** $a^{[2]} = \sigma(z^{[2]})$ * Note: $a^{[2]}$ is equal to the final prediction $\hat{y}$.

Dimensions Recap (Example)

- **Input x :** (3×1)
 - **Hidden Layer (4 nodes):**
 - $W^{[1]}$: (4×3)
 - $b^{[1]}$: (4×1)
 - **Output Layer (1 node):**
 - $W^{[2]}$: (1×4)
 - $b^{[2]}$: (1×1)
-

Vectorizing Across Multiple Examples

Motivation

In previous lessons, the forward propagation equations were derived for a **single training example** $x^{(i)}$.

- To process an entire training set of m examples, a naive implementation would use a `for` loop to iterate from $i = 1$ to m .
- **Goal:** Eliminate the explicit `for` loop by using matrix operations to process all m examples simultaneously, significantly improving computational efficiency.

Notation for Multiple Examples

- $x^{(i)}$: The i -th training example.
- $a^{[l](i)}$: The activation of layer l for the i -th training example.
- **Indices:**
 - **Square brackets** $[l]$: Refer to the layer number.
 - **Round brackets** (i) : Refer to the training example index.

The Vectorization Process

1. Constructing the Input Matrix X

Instead of processing individual vectors $x^{(i)}$, stack them as columns to form a matrix X .

- $X = [x^{(1)} \mid x^{(2)} \mid \dots \mid x^{(m)}]$
- Dimensions: $(n_x \times m)$, where n_x is the number of input features and m is the number of training examples.

2. Constructing Matrix Z and A

Similarly, the z and a vectors for each example are stacked horizontally to form matrices.

- $Z^{[1]} = [z^{1} \mid z^{[1](2)} \mid \dots \mid z^{[1](m)}]$
- $A^{[1]} = [a^{1} \mid a^{[1](2)} \mid \dots \mid a^{[1](m)}]$

3. Vectorized Equations

The `for` loop is replaced by the following four matrix equations:

1. **Layer 1 Linear Step:** $Z^{[1]} = W^{[1]}X + b^{[1]}$

2. **Layer 1 Activation:** $A^{[1]} = \sigma(Z^{[1]})$

3. **Layer 2 Linear Step:** $Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$

4. **Layer 2 Activation:** $A^{[2]} = \sigma(Z^{[2]})$

- *Note:* In Python/NumPy, adding b (a column vector) to a matrix triggers **broadcasting**, automatically adding the bias to every column.

Understanding Matrix Dimensions

It is crucial to understand what the rows and columns represent in these matrices (e.g., matrix $A^{[l]}$).

- **Horizontal Axis (Columns):** Index across **training examples** (1 to m).
 - Moving left to right scans through the dataset.
- **Vertical Axis (Rows):** Index across **nodes (hidden units)** in the layer.
 - Moving top to bottom scans through the neurons in that specific layer.

Example Interpretation:

- The value at the top-left corner of matrix $A^{[1]}$ corresponds to the activation of the **first hidden unit** on the **first training example**.
 - The value immediately below it corresponds to the **second hidden unit** on the **first training example**.
 - The value to the immediate right corresponds to the **first hidden unit** on the **second training example**.
-

Explanation for Vectorized Implementation

Justification for Vectorization

The validity of the vectorized approach relies on standard matrix multiplication properties.

- **Single Example Logic:**
 - For a single training example $x^{(1)}$, computing $z^{1} = W^{[1]}x^{(1)}$ results in a column vector.
 - Similarly, for $x^{(2)}$, $z^{[1](2)} = W^{[1]}x^{(2)}$ is another column vector.
- **Matrix Construction:**
 - The input matrix X is formed by stacking inputs as columns:
$$X = [x^{(1)} \mid x^{(2)} \mid x^{(3)} \dots].$$
- **Matrix Multiplication Result:**
 - Multiplying $W^{[1]}$ by X effectively multiplies $W^{[1]}$ by each column of X independently and places the results in corresponding columns.
 - $W^{[1]}X = [W^{[1]}x^{(1)} \mid W^{[1]}x^{(2)} \mid W^{[1]}x^{(3)}]$
 - This results in: $[z^{1} \mid z^{[1](2)} \mid z^{[1](3)}] = Z^{[1]}$
 - **Conclusion:** Stacking inputs horizontally results in outputs (Z and A) also being stacked horizontally.

Handling the Bias Term (Broadcasting)

- In the simplified justification above, the bias b was ignored ($b = 0$).
- **Python Broadcasting:** When adding the bias vector $b^{[1]}$ to the matrix $W^{[1]}X$, Python automatically "broadcasts" the vector.
 - This means $b^{[1]}$ is added individually to **every column** of the resulting matrix, ensuring the calculation $z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$ holds true for all examples simultaneously.

Consistency and Notation

By recognizing that the input features X can be denoted as the activations of the "zeroth" layer ($A^{[0]}$), a symmetric pattern emerges in the forward propagation equations.

- **Substitution:** Replace X with $A^{[0]}$.
 - **Layer 1:**
$$Z^{[1]} = W^{[1]}A^{[0]} + b^{[1]}$$
$$A^{[1]} = \sigma(Z^{[1]})$$
 - **Layer 2:**
$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$
$$A^{[2]} = \sigma(Z^{[2]})$$
 - **General Insight:** This demonstrates that neural networks, regardless of depth, repeat the same fundamental computation steps (Linear calculation $\rightarrow$ Activation) layer by layer.
-

Activation Functions

Overview

When building a neural network, a critical design choice is selecting the **activation function** ($g(z)$) for both hidden layers and the output layer. While the sigmoid function was used in initial examples, other non-linear functions often yield better performance.

1. Sigmoid Function

- **Formula:** $a = \sigma(z) = \frac{1}{1+e^{-z}}$
- **Range:** $[0, 1]$
- **Usage:**
 - **Output Layer:** Ideally suited for **binary classification** where the output must be a probability between 0 and 1.
 - **Hidden Layers: Avoid using sigmoid.** It is rarely used in modern hidden layers because the **Tanh** function is strictly superior.
- **Downside:**
 - **Vanishing Gradients:** When z is very large or very small, the slope

(derivative) of the function becomes close to zero. This significantly slows down gradient descent learning.

2. Tanh (Hyperbolic Tangent) Function

- **Formula:** $a = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- **Range:** $[-1, 1]$
- **Usage:** Preferred over sigmoid for **hidden layers**.
- **Advantages:**
 - **Zero-Centered:** Unlike sigmoid (which centers around 0.5), Tanh centers activations around 0. This mimics data centering (standardization), making learning easier for the subsequent layer.
- **Downside:** Like sigmoid, it suffers from the vanishing gradient problem at extreme values of z .

3. ReLU (Rectified Linear Unit)

- **Formula:** $a = \max(0, z)$
- **Range:** $[0, \infty)$
- **Usage:** The **default** choice for hidden layers in most modern neural networks.
- **Advantages:**
 - **Faster Learning:** The slope is 1 for all positive z , preventing the gradient from vanishing for a large portion of the input space.
 - **Efficiency:** computationally cheaper than Tanh or Sigmoid (no exponentials).
- **Derivative:**
 - If $z > 0$, slope = 1.
 - If $z < 0$, slope = 0.
 - If $z = 0$, the derivative is technically undefined, but in implementation, it is treated as either 0 or 1.

4. Leaky ReLU

- **Formula:** $a = \max(0.01z, z)$
- **Description:** A variation of ReLU that includes a slight slope (e.g., 0.01) for negative values of z .
- **Advantage:** Solves the issue where ReLU neurons become "dead" (zero gradient) for negative inputs.
- **Usage:** Often performs slightly better than ReLU, though standard ReLU

remains the most common industry standard.

Guidelines for Choosing Activation Functions

Context	Recommended Function
Binary Classification Output	Sigmoid (natural choice for 0/1 output)
Hidden Layers (Default)	ReLU (most common, fast learning)
Hidden Layers (Alternative)	Tanh (better than sigmoid, strictly)
Hidden Layers (Advanced)	Leaky ReLU (worth trying if ReLU stalls)

Key Takeaway: Deep learning is empirical. If you are unsure, start with **ReLU** for hidden layers. However, because different problems have different idiosyncrasies, it is often best to test multiple activation functions on a validation set to see which performs best for your specific application.

Why Do You Need Non-Linear Activation Functions?

The Problem with Linear Activation Functions

If a neural network uses **linear activation functions** (also called **identity activation functions**, where $g(z) = z$) for its hidden layers, the network loses its ability to compute complex, interesting functions.

Mathematical Proof

Consider a standard 2-layer network where the activation function $g(z) = z$ (linear).

- 1. **Layer 1 (Hidden):**
 $a^{[1]} = z^{[1]} = W^{[1]}x + b^{[1]}$
- 2. **Layer 2 (Output):**
 $a^{[2]} = z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$

If we substitute the equation for $a^{[1]}$ into the equation for $a^{[2]}$:

$$a^{[2]} = W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]}$$

$$a^{[2]} = (W^{[2]}W^{[1]})x + (W^{[2]}b^{[1]} + b^{[2]})$$

This simplifies to a standard linear equation:

$$a^{[2]} = W'x + b'$$

Key Takeaway

- **Composition of Linear Functions:** The composition of two linear functions is itself a linear function.
- **Collapse of Complexity:** If all hidden layers use linear activations, the entire network—no matter how deep (how many layers)—collapses into a standard linear model. It becomes no more expressive than standard logistic or linear regression.
- **Conclusion:** To learn complex patterns, hidden layers **must** use non-linear activation functions (ReLU, Tanh, Sigmoid, etc.).

The Single Exception: Output Layer

There is one specific scenario where a linear activation function ($g(z) = z$) is appropriate, but **only for the output layer**.

- **Regression Problems:** If the goal is to predict a real number (a continuous scalar value) rather than a classification probability.
 - *Example:* Predicting housing prices (where the output y can be any real number, potentially from 0 to millions).
 - **Constraint:** Even in this case, all **hidden layers** must still use non-linear activations (e.g., ReLU or Tanh) to ensure the network can learn non-linear relationships.
 - *Note:* If the output must be strictly non-negative (like housing prices), a ReLU activation at the output layer is often preferred over a purely linear one.
-

Derivatives of Activation Functions

Importance in Backpropagation

To implement backpropagation for a neural network, you must compute the **slope** (derivative) of the activation functions. The derivative of the activation function $g(z)$ with respect to z is denoted as $g'(z)$ or $\frac{d}{dz}g(z)$.

1. Sigmoid Activation Function

- **Function:** $g(z) = \sigma(z) = \frac{1}{1+e^{-z}}$
- **Derivative Formula:**
 $g'(z) = g(z)(1 - g(z))$
- **Implementation Note:** Since $a = g(z)$, the derivative can be efficiently computed as:
 $g'(z) = a(1 - a)$
- **Sanity Check:**
 - **Large z (e.g., 10):** $a \approx 1$. Derivative $\approx 1(1 - 1) \approx 0$. (Correct: slope is flat).
 - **Small z (e.g., -10):** $a \approx 0$. Derivative $\approx 0(1 - 0) \approx 0$. (Correct: slope is flat).
 - $z = 0$: $a = 0.5$. Derivative $= 0.5(1 - 0.5) = 0.25$. (Max slope).

2. Tanh Activation Function

- **Function:** $g(z) = \tanh(z)$
- **Derivative Formula:**
 $g'(z) = 1 - (g(z))^2$
- **Implementation Note:** Since $a = g(z)$, the derivative is:
 $g'(z) = 1 - a^2$
- **Sanity Check:**
 - **Large/Small z :** $a \approx 1$ or -1 . Derivative $\approx 1 - (1)^2 = 0$.
 - $z = 0$: $a = 0$. Derivative $= 1 - 0 = 1$.

3. ReLU (Rectified Linear Unit)

- **Function:** $g(z) = \max(0, z)$

- **Derivative Formula:**

$$g'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z > 0 \end{cases}$$

- **Edge Case ($z = 0$):**

- Technically, the derivative is **undefined** (non-differentiable) at exactly $z = 0$.
- **In Practice:** In software implementation, you can arbitrarily set the derivative to either **1** or **0**. The probability of z being exactly $0.000000\dots$ is vanishingly small, so this choice does not affect the optimization algorithm.

4. Leaky ReLU

- **Function:** $g(z) = \max(0.01z, z)$

- **Derivative Formula:**

$$g'(z) = \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z > 0 \end{cases}$$

- **Edge Case ($z = 0$):** Similar to standard ReLU, the gradient is undefined at 0. In code, it is safe to set it to either 0.01 or 1.
-

Gradient Descent for Neural Networks

Overview

This section outlines the specific equations required to implement gradient descent for a neural network with a **single hidden layer**. The process involves Forward Propagation to get predictions, followed by Backpropagation to compute derivatives, and finally updating the parameters.

Network Architecture & Parameters

- **Structure:** 2-Layer Network (Input Layer → Hidden Layer → Output Layer).
- **Notation:**
 - $n_x = n^{[0]}$: Number of input features.
 - $n^{[1]}$: Number of hidden units.
 - $n^{[2]}$: Number of output units (1 for binary classification).
 - m : Number of training examples.
- **Parameter Dimensions:**
 - $W^{[1]}$: $(n^{[1]} \times n^{[0]})$
 - $b^{[1]}$: $(n^{[1]} \times 1)$
 - $W^{[2]}$: $(n^{[2]} \times n^{[1]})$
 - $b^{[2]}$: $(n^{[2]} \times 1)$

The Gradient Descent Algorithm

1. Forward Propagation

Calculate the predictions ($A^{[2]}$) across all m training examples.

1. $Z^{[1]} = W^{[1]}X + b^{[1]}$
2. $A^{[1]} = g^{[1]}(Z^{[1]})$ (where $g^{[1]}$ is the hidden layer activation, e.g., Tanh or ReLU)
3. $Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$
4. $A^{[2]} = \sigma(Z^{[2]})$ (Sigmoid for binary classification output)

2. Backpropagation (Computing Derivatives)

These 6 equations compute the gradients required for updates. The derivatives are calculated **right-to-left**.

- **Layer 2 (Output Layer):**
 1. $dZ^{[2]} = A^{[2]} - Y$
 2. $dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$
 3. $db^{[2]} = \frac{1}{m} \text{np.sum}(dZ^{[2]}, \text{axis} = 1, \text{keepdims}=\text{True})$
- **Layer 1 (Hidden Layer):**
 4. $dZ^{[1]} = (W^{[2]T} dZ^{[2]}) * g'^{[1]}(Z^{[1]})$
Note: The $*$ represents **element-wise product**.

* $g'^{[1]}$ is the derivative of the hidden activation function.

$$5. dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$6. db^{[1]} = \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis} = 1, \text{keepdims}=\text{True})$$

3. Parameter Updates

Update the parameters using the calculated gradients and the learning rate α .

- $W^{[1]} = W^{[1]} - \alpha \cdot dW^{[1]}$
- $b^{[1]} = b^{[1]} - \alpha \cdot db^{[1]}$
- $W^{[2]} = W^{[2]} - \alpha \cdot dW^{[2]}$
- $b^{[2]} = b^{[2]} - \alpha \cdot db^{[2]}$

Implementation Details (Python/NumPy)

- **np.sum & keepdims=True:**
 - When computing db (summing across examples), use `axis=1` to sum horizontally.
 - **Crucial:** Set `keepdims=True` to prevent Python from outputting a "rank-1 array" (e.g., shape `(n, )`). This ensures the output remains a proper column vector (shape `(n, 1)`), which prevents broadcasting errors in subsequent steps.
 - *Alternative:* You can manually `reshape()` the output if you forget `keepdims`.
-

Backpropagation Intuition (Optional)

Logistic Regression Recap

To understand neural network backpropagation, it helps to review the simpler case of logistic regression.

- **Forward Pass:** Compute $z \rightarrow a \rightarrow \text{Loss}(a, y)$.
- **Backward Pass:** Compute derivatives in reverse: $da \rightarrow dz \rightarrow dw, db$.
- **Chain Rule Intuition:**
 - $dz = \frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot \frac{da}{dz}$
 - Since $a = g(z)$, this simplifies to $dz = da \times g'(z)$.

- This effectively collapses the step of computing da first; we often calculate dz directly as $a - y$.

2-Layer Neural Network Derivation

For a neural network with one hidden layer, the computation is similar but performed twice (Layer 2 $\rightarrow$ Layer 1).

Forward Path

1. $z^{[1]} \rightarrow a^{[1]}$ (Hidden Layer)
2. $z^{[2]} \rightarrow a^{[2]}$ (Output Layer)
3. Compute Loss.

Backward Path (Layer 2 - Output)

We skip explicitly computing $da^{[2]}$ and go straight to $dz^{[2]}$.

- $dz^{[2]}: a^{[2]} - y$ (Same as logistic regression).
- $dW^{[2]}: dz^{[2]} a^{[1]T}$
 - Note: We use $a^{[1]}$ here because the input to Layer 2 is the activation of Layer 1. The transpose ensures dimensions match.
- $db^{[2]}: dz^{[2]}$

Backward Path (Layer 1 - Hidden)

We propagate the error from Layer 2 back to Layer 1.

- $dz^{[1]}: W^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$
 - The $W^{[2]T} dz^{[2]}$ term represents the error flowing back through the weights.
 - The $*$ represents **element-wise multiplication**.
 - $g^{[1]'}$ is the derivative of the hidden layer's activation function.
- $dW^{[1]}: dz^{[1]} x^T$
 - Note: x is the input to Layer 1 (also denoted as $a^{[0]}$).
- $db^{[1]}: dz^{[1]}$

Dimensionality Analysis (Sanity Check)

Checking matrix dimensions is a powerful way to verify backpropagation equations and eliminate bugs.

- **General Rule:** A variable and its derivative always have the same dimensions (e.g., W and dW are both $n^{[l]} \times n^{[l-1]}$).
- **Checking $dz^{[1]}$:**
 - Formula: $W^{[2]T} dz^{[2]} * g'(z^{[1]})$
 - Dimensions: $(n^{[1]} \times n^{[2]}) \times (n^{[2]} \times 1) \rightarrow (n^{[1]} \times 1)$
 - This matches the dimension of $z^{[1]}$, confirming the matrix multiplication is valid.

Vectorized Backpropagation Equations

To implement this efficiently across m training examples, we stack examples as columns in matrices (X, A, Z).

Layer 2 (Output)

1. $dZ^{[2]} = A^{[2]} - Y$
2. $dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$
3. $db^{[2]} = \frac{1}{m} \text{np.sum}(dZ^{[2]}, \text{axis} = 1, \text{keepdims}=\text{True})$

Layer 1 (Hidden)

4. $dZ^{[1]} = (W^{[2]T} dZ^{[2]}) * g^{[1]'}(Z^{[1]})$
 5. $dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$
 6. $db^{[1]} = \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis} = 1, \text{keepdims}=\text{True})$
-

Random Initialization

The Problem with Zero Initialization

Initializing weights to zero works for Logistic Regression, but **it fails completely for Neural Networks**.

- **Symmetry Problem:** If you initialize the weight matrix W to all zeros:

- Every node in the hidden layer receives the same input and has the same parameters.
- Consequently, every hidden unit computes the **exact same activation** ($a_1^{[1]} = a_2^{[1]} = \dots$).
- During backpropagation, the derivatives for all hidden units will be identical ($dz_1^{[1]} = dz_2^{[1]}$).
- **Result:** The hidden units update identically in every iteration. No matter how long you train, all hidden units remain symmetric and compute the exact same function. The network effectively behaves as if it has only **one** hidden unit per layer.

The Solution: Random Initialization

To break this symmetry, you must initialize the weights randomly.

- **Weights (W):** Initialize with small random numbers (e.g., from a Gaussian distribution).
- **Biases (b):** It is safe to initialize biases to **zeros**. As long as W is random, symmetry is broken, and hidden units will learn distinct features.

Python Implementation

```
1 # Example for layer 1
2 w1 = np.random.randn((2, 2)) * 0.01
3 b1 = np.zeros((2, 1))
```

Why Initialize with *Small* Numbers (0.01)?

We typically multiply the random numbers by a small scalar (like 0.01) to keep the weights small.

- **Reasoning:**
 1. Recall $z^{[1]} = W^{[1]}x + b^{[1]}$.
 2. If W is large, z will likely be very large (positive or negative).
 3. If using **Sigmoid** or **Tanh** activation functions, large z values push the activation to the "flat" parts of the curve (saturation regions where output is near -1, 0, or 1).
 4. In these regions, the **slope (derivative) is very close to zero**.
 5. **Consequence:** Gradient descent becomes extremely slow because the updates are tiny.

- **Note:** For very deep networks, the constant 0.01 might need to be adjusted (a topic for future lessons), but the principle of keeping weights relatively small remains.
-